1 Introduction

Let random variables be denoted by A, R, R_t, A_t and their realizations in lower-case. All sets will be: $\mathbb{S}$ or similar.
RL $\stackrel{\text{def}}{=}$ Learning to solve sequential decision problems via repeated interaction with environment
RL Problem $\stackrel{\text{def}}{=}$ Learning a high-value policy by interacting with an MDP.
Reward Hypothesis $\stackrel{\text{def}}{=}$ All goals can be described by the maximization of the expected value of cumulative scalar rewards.
Policy $\stackrel{\text{def}}{=}$ State to Action Map
Reward Signal $\stackrel{\text{def}}{=}$ Defines goal of RL. At each time step, environment sends the agent a reward.
Value Function $\stackrel{\text{def}}{=}$ Value of a state is the total amount of reward an agent can expect to accumulate in the future, starting from the state
Supervised $\stackrel{\text{def}}{=}$ Discover unknown function $f(x) = y$ given examples $(x, y = f(x))$ (RL does not get correct actions provided)
Unsupervised $\stackrel{\text{def}}{=}$ Discover hidden structure in data $x_1, x_2, x_3, \ldots$ (no y given) (Rl gets a reward signal to inform correct action)
Key Challenges in RL:
1. Unknown Environment (how do actions affect state and rewards)
2. Exploration-Exploitation Dilemma (try new a or stick to best a)
3. Delayed Rewards (an a may affect future rewards)

2 Multi-Arm Bandits

Definition (Think MDP with only one state):

> **Given**: a set of k actions, A, number of rounds T:
> Repeat for t in T rounds:
> 1. Algorithm selects arm A_t in A
> 2. Algorithm observes reward $R_t \in [0, 1]$
>
> **Goal**: maximize expected total reward.

Note: No role of state in MAB + over set number T!
Value of Arm a:

$$q_*(a) \stackrel{\text{def}}{=} \mathbb{E}[R_t \mid A_t = a] \text{ (cannot get directly)}$$

Sample Average Estimate of Action-Value:

$$Q_{t(a)} \stackrel{\text{def}}{=} \frac{1}{N_t(a)} * \sum_{i=1}^{t-i} R_i * \mathbb{1}_{A_t=a}$$

(sum of rewards when a taken/num times a taken, prior to t)

Incremental Update:

$$Q_{t+1} \stackrel{\text{def}}{=} Q_t + \frac{1}{n}[R_n - Q_n]$$

Standard Form of Update:

NewEstimate ← OldEstimate + StepSize[Target - OldEstimate]

Error $\stackrel{\text{def}}{=}$ Target - OldEstimate

A simple bandit algorithm

Initialize, for $a = 1$ to k:
$Q(a) \leftarrow 0$
$N(a) \leftarrow 0$

Loop forever:
$A \leftarrow \begin{cases} \text{argmax}_a\, Q(a) & \text{with probability } 1-\varepsilon \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$ (breaking ties randomly)
$R \leftarrow bandit(A)$
$N(A) \leftarrow N(A) + 1$
$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

The action selection above is ε-greedy exactly, which is useful when there are noisy rewards and no one action is always best. The above alg only applies to **stationary problems** (where reward probabilities don't change).

2.1 Non-Stationary MAB

NS = non-stationary
Incremental Update :

$$Q_{t+1} \stackrel{\text{def}}{=} Q_t + \alpha[R_n - Q_n]$$

where α is the step-size parameter (learning rate) given by $\alpha_t(a)$

Standard Stochastic Approximation Conditions (SSAC):
Estimates Q_n converge to q_* with Pr=1 if:

$$\sum_{n=1}^{\infty} \alpha_n(a) \to \infty \text{ and } \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

However, in **NS** problems, $\alpha_n(a)$ that do converge (e.g. $\frac{1}{n}$), do so very slowly and need tuning to speed up convergence. It is desirable to use $\alpha_n(a)$ that do not meet the conditions (e.g. constant α) in **NS** problems.

2.2 Increasing Exploration in MAB

1. **Optimistic Initial Values**

The initial action-value estimate $Q_1(a)$ *biases* MAB. Setting a high initial action value instead of just 0 encourages exploration. The learner is 'disappointed' by the lower actual reward and switches to other actions, but this results in several actions being tried before value estimates converge. ✓ Simple ✗ Only useful in stationary problems as all exploration is only ever done initially.
2. **Upper-Confidence-Bound (UCB) Action Selection**

$$A_t \stackrel{\text{def}}{=} \text{argmax}_a \left[Q_t(a) + c\sqrt{\ln \frac{t}{N_t}(a)} \right]$$

$c > 0$ and controls the degree of exploration. Here actions are picked by highest bound. ✓ Actions picked because either algorithm is uncertain about an action (never picked or picked long ago, scaling with unbounded $\ln t$) or it can exploit a high value action. Hence all actions eventually get selected but actions with lower value estimates or those selected frequently, get selected with a decreasing frequency over time. ✓ Performs better than ε-greedy.
3. **Gradient-Bandit Algorithms**

This is a policy-based approach and does not focus on a value function.
Preference: Each action is assigned a scalar $H_t(a) \in \mathbb{R}$.
A differentiable policy $\pi_t(a \mid \theta)$, where $\theta \in \mathbb{R}^d$ (policy parameters), is utilized such that *gradient ascent* could be performed:

$$\theta_{t+1} \stackrel{\text{def}}{=} \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}[R_t]$$

The policy itself could be represented by a softmax distribution

$$\pi_{t(a)} \stackrel{\text{def}}{=} \Pr\{A_t = a\} \stackrel{\text{def}}{=} \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}}$$

Preference Update:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha\big(R_t - \overline{R_t}\big)(1 - \pi_t(A_t))$$

where $\overline{R_t} = \frac{1}{t}\sum_{i=1}^{t} R_i$ is the average reward.

3 Markov-Decision Processes

MDP: Consists of the **state** (unlike MAB), action and reward space + environment dynamics $p(s', r \mid s, a)$. Finite MDP if S, A, R finite. The MDP is controlled with a stochastic or deterministic policy π.
Environment Dynamics:

$$p(s' \mid s, a) \stackrel{\text{def}}{=} \sum_{r \in R} p(s', r \mid s, a)$$

$$r(s, a) \stackrel{\text{def}}{=} \sum_{r \in R, s' \in S} r * p(s', r \mid s, a)$$

$$r(s, a, s') \stackrel{\text{def}}{=} \sum_{r \in R} r * \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}$$

Trajectory: sequence of states, actions and rewards:
$S_0, A_0, R_1, S_1, A_1, R_2, \ldots$
Markov Property: Future state and reward are independent of past states and actions, given the current state and action.
Markov Chain: MDP w/ a deterministic policy
Return (un-discounted): $G_t \stackrel{\text{def}}{=} R_{t+1} + R_{t+2} + \ldots = R_{t+1} + G_{t+1}$
Episode: Particular sequence of agent-environment interaction that ends w/ a terminal state at time step T. Tasks with episodes are called **Episodic** and distinguish between S (non-terminal states) and S^+ (all states).
Discounting: In order to accommodate tasks that continue without limit (called *continuing* or *non-episodic*), a **discount rate** γ is introduced:

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma G_{t+1} \stackrel{\text{def}}{=} \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \text{ where } 0 \le \gamma \le 1$$

For terminating states, $G_T = 0$ by convention. The sum itself is finite for $\gamma < 1$ and bounded rewards $R_t \le r_{\max}$:

$$\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \le r_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{r_{\max}}{1-\gamma}$$

Absorbing State: Terminal state that transitions into itself and gives reward 0.
Bellman Equations: For *state-value function* it is

$$\begin{aligned} v_\pi(s) &\stackrel{\text{def}}{=} \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s',r} [r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \\ &= \sum_a \pi(a \mid s) \sum_{s',r} [r + \gamma v_\pi(s')] \end{aligned}$$

This is a direct result of the Markov Property! The *action-value function*:

$$q_\pi(s, a) \stackrel{\text{def}}{=} \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$$

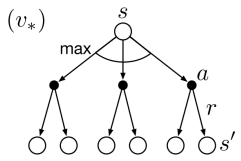
Optimal Policy: A policy π is optimal if $v_\pi(s) \ge v_{\pi'}(s), \forall s \in S$ and $\forall \pi' \neq \pi$. Any π that is optimal is denoted π_* and they all share the same value functions v_* and q_*. In particular:

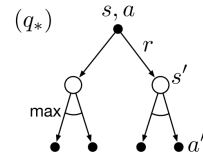
$$v_*(s) \stackrel{\text{def}}{=} \max_\pi v_\pi(s) \text{ and } q_*(s, a) \stackrel{\text{def}}{=} \max_\pi q_\pi(s, a), \forall s, a$$

Bellman Optimality Equations:

$$\begin{aligned} v_*(s) &\stackrel{\text{def}}{=} \max_{a \in A} q_{\pi_*}(s, a) \\ &= \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_*(s')] \end{aligned}$$

$$\begin{aligned} q_*(s, a) &\stackrel{\text{def}}{=} \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\Big] \\ &= \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \max_{a'} q_*(s', a')\Big] \end{aligned}$$





Note:
1. The value function v (and q) can be computed as a unique solution to |S| Bellman Equations (one for each state)
2. The optimal v_* could also be computed uniquely with a system of Bellman Optimality Equations, but due to the *max* operator, the system consists of non-linear equations.
3. Either above are rarely done as it is computationally intensive and scales with the number of states. Moreover, the true dynamics of the environment is rarely accurately known.

4 Dynamic Programming Methods

DP Methods compute optimal policies given a perfect model (MDP), thereby are a family of **planning** algorithms. **Planning** refers to the problem of learning a policy given a model. **DP** turn the Bellman Equations into update rules and all updates done in **DP** are called *expected* updates because they are based on an expectation over all possible next states rather than on a sample next state.
Policy Evaluation (Prediction Problem): Computing v_π from π.
Policy Improvement: Improving π from v_π. Generally by making it greedy with respect to the value function of the original policy.

4.1 Policy Evaluation

The v_π Bellman Equation is turned into an update rule:

$$\begin{aligned} v_{k+1}(s) &\stackrel{\text{def}}{=} \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_k(s')] \end{aligned}$$

It is proven that v_k converges to v_π as $k \to \infty$.

4.2 Policy Improvement

Policy Improvement Theorem: Let π and π' be two deterministic policies such that for all $s \in S$, $q_\pi(s, \pi'(s)) \ge v_\pi(s)$. Then the policy π' must be as good as, or better than π - i.e. $v_{\pi'}(s) \ge v_\pi(s), \forall s \in S$.
We can define a greedy policy as follows:

$$\begin{aligned} \pi'(s) &= \text{argmax}_a q_\pi(s, a) \\ &= \text{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \text{argmax}_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \end{aligned}$$

Once π' is as good as π, $v_\pi = v_{\pi'}$. It follows that:

$$\begin{aligned} v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_{\pi'}(s')] \end{aligned}$$

Where the above is exactly the Bellman Optimality Equation, so $v_{\pi'}$ must be v_* and π' must be π_*!
Stochastic Policies? Only deterministic policies discussed before but all also applies to stochastic policies $\pi(a \mid s)$. In the case of multiple actions having a maximum value, the stochastic policy could give an equal portion probability to all instead of selecting only one.

4.3 Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathbb{S}$; $V(terminal) \doteq 0$
2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathbb{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))[r + \gamma V(s')]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathbb{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \text{argmax}_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Note:
1. Only ever sweep through non-terminal states.
2. Implicit is that we are updating incrementally, so V_{k+1} on LHS!
3. Ties broken arbitrarily when making policy greedy (if deterministic)
4. PI often converges in a few iterations + optimality guaranteed!

4.4 Value Iteration

Policy evaluation in *policy iteration* requires multiple sweeps through the state set rather than being done iteratively. **Value iteration** truncates *policy evaluation* (does not wait for it to converge) to a single sweep without losing converge guarantees. The combined policy improvement and truncated policy evaluation iterative step is:

$$v_{k+1}(s) \stackrel{\text{def}}{=} \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a]$$
$$= \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_k(s')], \forall s \in S$$

The sequence $\{v_k\}$ is shown to converge to v_* under the same conditions that guarantee the existence of v_*.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$

General Notes on the DP Methods:

1. *Value Iteration* uses the Bellman optimality equation for the value function while *Policy Iteration* uses just the Bellman equation.
2. Both methods formally require an infinite number of iterations to converge exactly to v_*, though in practice the iteration is stopped when changes are small between iterations.
3. Both converge to an optimal policy for discounted finite MDPs.
4. The entire idea of carrying out policy evaluation and improvement in a cycle is a general idea called **Generalized Policy Iteration** (GPI).
5. **DP** assumes that environment dynamics are completely known!
6. **DP** methods, by using Bellman (Optimality) Equations, **bootstrap** (create estimates based on other estimates).

Asynchronous DP:

1. Asynchronous **DP** methods evaluate and improve policy on subset of states instead of sweeping the entire state set. These algorithms are in-place iterative **DP** algorithms that can update the values of states in any order whatsoever. Convergence is guaranteed as long as all states continue to be updated an infinite number of times
3. Greater flexibility to choose best states to updates. For example, now real-time interaction is possible as the **DP** algorithm can be run *at the same time that an agent is actually experiencing the MDP*. Updates only need to be performed on states actually visited.
4. Can perform updates in parallel across multiple processors

5 Monte Carlo Methods

MC Methods $\stackrel{\text{def}}{=}$ methods that learn the value function based on experience. These do not require a complete model, only require sampled episodes. Put another way, MC is a way of solving the RL problem using average sample returns (*returns instead of rewards as we refer to rewards across episodes*).
Experience: entire episodes $E^i =< S_0^i, A_0^i, R_1^i, S_1^i, A_1^i, ... >$
Assumptions of MC Algorithms:

1. Episodic tasks
2. Only on a completion of an episode are value estimates and policy updated (hence MC is incremental across episodes but not time steps)

5.1 MC Prediction (Policy Evaluation)

With MC, the value of a state is estimated from experience by simply averaging all returns observed after visits to that state. As more results are observed, the average converges to the expected value.
Visit to s: Each occurrence of state s in an episode.
First-Visit MC: estimate $v_\pi(s)$ as the average returns following the first visit to s across episodes.
Every-Visit MC: estimate $v_\pi(s)$ as the average returns following all visits to s across episodes.
The value function is estimated as below:

$$v_\pi(s) \stackrel{\text{def}}{=} \mathbb{E}_\pi\left[\sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1} \mid S_t = s\right] \approx \frac{1}{|\,\varepsilon(s)\,|} \sum_{t_i \in \varepsilon(s)} \sum_{k=t_i}^{T_i - 1} \gamma^{k-t_i} R_{k+1}^i$$

where

$$\varepsilon(s) := \begin{cases} \text{contains first time } t_i \text{ for which } S_{t_i}^i = s \text{ in } E^i \text{ if First-Visit} \\ \text{contains all times } t_i \text{ for which } S_{t_i}^i = s \text{ in } E^i \text{ if Every-Visit} \end{cases}$$

Both methods converge to $v_\pi(s)$ as $|\,\varepsilon(s)\,| \to \infty$, i.e. as the number of visits to all states reaches infinity.

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
Append G to $Returns(S_t)$
$V(S_t) \leftarrow$ average$(Returns(S_t))$

Note:

1. Every-Visit is the above without the *"Unless S_t appears ..."* statement.
2. Unlike DP, estimates for each state in MC are independent and there is no bootstrapping involved.
3. Actual experience is learned from and not just simulation
4. If only the value of a particular state is required, sample episodes could be generated starting from that state only to learn from.

5.2 MC Estimate of Action Value

The state-value function alone is not sufficient without a model - there are no environment dynamics available. As such, it is necessary to look at action-value instead. However, now both a start state and action must be specified to begin an episode, and this may not be sufficient in ensuring that all state-action pairs are visited (infinitely for convergence).
Exploring Starts (ES): every (s, a) pair has a non-zero probability of being the starting pair of an episode. (*maintaining exploration*)
Assumption of ES: all state-action pairs will be visited an infinite number of times in the limit of an infinite number of episodes.

5.3 MC Control (GPI)

With alternating policy evaluation and improvement, under the assumption of **ES** and infinite episodes, the approximation of a value function and policy are guaranteed to converge to q_π and π.
As before, policy improvement is done by making the policy greedy w.r.t. the action-value function $q_\pi(a, s) \forall a, s$. The *policy improvement theorem* applies as discussed before.

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
Append G to $Returns(S_t, A_t)$
$Q(S_t, A_t) \leftarrow$ average$(Returns(S_t, A_t))$
$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

Even-though convergence requires an ∞ number of episodes, in practice the *MC ES* is only ran till a given performance threshold is met.

5.4 On-Policy MC Prediction w/ ε-soft policy

On-Policy: attempt to evaluate or improve the policy that is used to make decisions (i.e. from which experience is generated).
Off-Policy: evaluate or improve a *target policy* π different from that used to generate the data (behavioral policy b).
The major issue with *MC ES* is the assumption of *ES*. Instead of *ES*, another approach for exploration is to use a stochastic policy instead.
ε-soft policy: a policy where $\pi(a \mid s) > 0, \forall s, a..$
Now an **ε-soft policy** can be expressed as:

$$\pi(a \mid s) := \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\,\mathcal{A}(S_t)\,|} & \text{if } a = A^* \\ \frac{\varepsilon}{|\,\mathcal{A}(S_t)\,|} & \text{if } a \neq A^* \end{cases}$$

Each action has a minimum probability of $\frac{\varepsilon}{|\,\mathcal{A}(S_t)\,|}$ regardless of $\varepsilon > 0$. For *policy improvement*, note that now it is not possible to simply pick the greedy action as the policy must remain *soft*. Luckily, GPI does not require that the ε-soft policy π be taken all the way to a greedy policy, but only moved toward one.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$
Initialize:
$\pi \leftarrow$ an arbitrary ε-soft policy
$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):
Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
Append G to $Returns(S_t, A_t)$
$Q(S_t, A_t) \leftarrow$ average$(Returns(S_t, A_t))$
$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)
For all $a \in \mathcal{A}(S_t)$:
$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$

To prove that an ε-greedy policy π' is an improvement over any ε-soft policy π, consider this proof of the *policy improvement theorem*:

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a \mid s) q_\pi(s, a)$$
$$= \frac{\varepsilon}{|\,\mathcal{A}(s)\,|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \max_a q_\pi(s, a)$$
$$\geq \frac{\varepsilon}{|\,\mathcal{A}(s)\,|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \sum_a \frac{\pi(a \mid s) - \frac{\varepsilon}{|\,\mathcal{A}(s)\,|}}{1 - \varepsilon} q_\pi(s, a)$$
$$= \sum_a \pi(a \mid s) q_\pi(s, a)$$
$$= v_\pi(s)$$

Step beginning $\geq$ follows from $\pi(a \mid s) - \frac{\varepsilon}{|\,\mathcal{A}(S_t)\,|} = 0, \forall a \neq A^*$.

5.5 Off-Policy MC Prediction w/ Importance Sampling

The *on-policy* approach before is a compromise, as it does not learn the optimal policy, but a near-optimal policy that still explores. The *off-policy* approach maintains a **target policy** π that becomes optimal and a **behavioral policy** b which only generates data and explores.
Assumption of Coverage: Every action taken under π is also taken at least occasionally under b. That is, $\pi(a \mid s) > 0$ implies $b(a \mid s) > 0$.
In order to meet the assumption, b must be stochastic and π could be either, but is generally a deterministic greedy policy.
Importance Sampling (IS): $\stackrel{\text{def}}{=}$ Estimating expected values under one distribution given samples from another.
Trajectories Given a starting state S_t, the probability of subsequent state-action trajectory under any π is:

$$\Pr\{A_t, S_{t+1}, A_{t+1}, ... \mid S_t, A_{t:T-1} \sim \pi\}$$
$$= \pi(A_t \mid S_t) p(S_{t+1} \mid S_t, A_t) \pi(A_{t+1} \mid S_{t+1}) ...$$
$$= \prod_{k=t}^{T-1} \pi(A_k \mid S_k) p(S_{k+1} \mid S_k, A_k)$$

IS is necessary as we estimate values under π given samples from b.
Importance Sampling Ratio: The relative probability of a trajectory occurring under the target and behavior policies. It is given by:

$$\rho_{t:T-1} \stackrel{\text{def}}{=} \frac{\prod_{k=t}^{T-1} \pi(A_k \mid S_k) p(S_{k+1} \mid S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k \mid S_k) p(S_{k+1} \mid S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

The **IS** ratio only depends on the two policies and not $p(s', r \mid s, a)$. Therefore, it is possible to transform the expected return under b as follows:

$$\mathbb{E}_b[\rho_{t:T-1} G_t \mid S_t = s]$$
$$= \sum_{E: S_t = s} \left[\prod_{k=t}^{T-1} b(A_k \mid S_k) p(S_{k+1} \mid S_k, A_k)\right] \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)} G_t$$
$$= \sum_{E: S_t = s} \left[\prod_{k=t}^{T-1} \pi(A_k \mid S_k) p(S_{k+1} \mid S_k, A_k)\right] G_t = v_\pi(s)$$

The approximation for the value functions is now just:

$$v_\pi(s) \approx \eta^{-1} \sum_{t_i \in \varepsilon(s)} p_{t_i:T_i} G_{t_i}^i \textbf{ and } q_\pi(s, a) \approx \eta^{-1} \sum_{t_i \in \varepsilon(s,a)} p_{t_i+1:T_i} G_{t_i}^i$$

Where $\varepsilon(s)/\varepsilon(s, a)$ denote the first/all times t_i for which the $s/(s, a)$ occur in E^i. Note the $t_i + 1$ for action-value as the input a is not present in the estimated trajectory from (s, a). The denominator η describes the which type of **IS** average is required:

1. **Ordinary IS**: $\eta = |\,\varepsilon(s)\,|$ or $\eta = |\,\varepsilon(s, a)\,|$
2. **Weighted IS**:

$$\eta = \sum_{t_i \in \varepsilon(s)} p_{t_i:T_i} \textbf{ or } \eta = \sum_{t_i \in \varepsilon(s)} p_{t_i+1:T_i}$$

Comparisons of **IS** type:

1. For when there is only a single-return under First-Visit MC, the *ordinary* case gives v_π (unbiased) whereas *weighted* gives the observed return under b (**IS** ratio cancels out), which is biased!
2. The *ordinary* case's variance is unbounded on a single-return First-Visit MC, but the *weighted* version is bounded to one.
3. On Every-Visit MC, both **IS** types are biased.

5.6 Incremental Implementation of Off-Policy MC

MC is incremental across episodes and not time steps. For the *ordinary* case, it is only a matter of scaling w.r.t. $\varepsilon(s)/\varepsilon(s, a)$. For the *weighted* case, the following estimate is needed:

$$V_n \stackrel{\text{def}}{=} \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}, \text{ where } W_k = \rho_{t_i:T(t_i)-1} \quad \text{and} \quad n \geq 2$$

In addition to V_n, a cumulative sum C_n of the weights given in the first n returns is maintained to provide the following incremental updates:

$$V_{n+1} \stackrel{\text{def}}{=} V_n + \frac{W_n}{C_n}[G_n - V_n] \text{ for } n \geq 1 \text{ as } V_1 \text{ is arbitrary}$$
$$C_{n+1} \stackrel{\text{def}}{=} C_n + W_{n+1} \text{ where } C_0 \stackrel{\text{def}}{=} 0$$

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
$Q(s, a) \in \mathbb{R}$ (arbitrarily)
$C(s, a) \leftarrow 0$

Loop forever (for each episode):
$b \leftarrow$ any policy with coverage of π
Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
$W \leftarrow 1$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
$G \leftarrow \gamma G + R_{t+1}$
$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$
$W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$
  $\pi(s) \leftarrow \text{argmax}_a\, Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
  $b \leftarrow$ any soft policy
  Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\left[G - Q(S_t, A_t)\right]$
    $\pi(S_t) \leftarrow \text{argmax}_a\, Q(S_t, a)$ (with ties broken consistently)
    If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$

× A potential problem is that this method learns only from the tails of episodes, when all of the remaining actions in the episode are greedy.
× If nongreedy actions are common, then learning will be slow, particularly for states appearing in the early portions of long episodes.

General Notes about Off-Policy

1. As data comes from a different policy, convergence is of greater variance and slower than on-policy.
2. Off-policy methods are more "powerful" and general. On-policy is a special case where the target and behavioral policy are the same.
3. Ordinary importance sampling produces unbiased estimates, but has larger, possibly infinite, variance, whereas weighted importance sampling always has finite variance and is preferred in practice

6 Temporal-Difference Learning

TD sits between DP and MC method. Like MC methods, it learns directly from raw experience without knowing any environment dynamics. Like DP, it updates its estimates based in part on other learned estimates, without waiting for a final outcome (**bootstraps**).

6.1 TD-Prediction (Policy Evaluation)

Whereas MC methods wait till the end of an episode before updating estimates (only then is G_t known), TD methods only need to wait until the next time step. At time $t+1$, a target is formed using R_{t+1} and estimate for $V(S_{t+1})$. The **TD(0) update** has the form:

$$V(S_t) \leftarrow V(S_t) + \alpha\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

TD(0) is one-step TD and it can be used to evaluate a policy as follows:

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0,1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\left[R + \gamma V(S') - V(S)\right]$
    $S \leftarrow S'$
  until S is terminal

6.2 TD Advantages and Comparisons

TD estimates as both DP and MC do.

1. **MC** estimates as the expected value $\mathbb{E}_\pi[G_t \mid S_t = s]$ is not known and a sample return is used in place of a real return.
2. **DP** estimates not because of expected values (it is assumed it has a full model), but as it uses V to estimate the true v_π by bootstrapping.
3. **TD** estimates as it samples expected values and bootstraps to find v_π

Both **MC** and **TD** use *sample updates* as opposed to *expected* updates like **DP** since they both use a single sample successor during updates.

TD(0) Error:

$$\delta_t \stackrel{\text{def}}{=} R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

MC errors can be given in terms of **TD(0)** errors as:

$$G_t - V(S_t) \stackrel{\text{def}}{=} \sum_{k=t}^{T-1} \gamma^{k-t} \delta_k$$

TD(0) Convergence to v_π with prob=1 if: All states visited infinitely often and **SSAC** met (see 2.1)

TD Advantages

1. Like **MC**, a full model is not required
2. Unlike **MC**, TD can be fully incremental (as it bootstraps) and thus requires less memory and computation
3. **TD(0)** is shown to converge with any fixed policy π and in practice, it is shown to converge faster than **MC**.

Method Summary

Method	Model-Free?	Bootstrap?
DP	No	Yes
MC	Yes	No
TD	Yes	Yes

6.3 On-Policy TD(0) Control: SARSA or SARSA(0)

Sarsa uses the quintuples $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ (hence its name) for updates and its incremental update rule for action-value is given as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

This update is done after every transition from a non-terminal state and if S_{t+1} were terminal then $Q(S_{t+1}, A_{t+1})$ is defined as zero.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$
Initialize $Q(s,a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S', A') - Q(S,A)\right]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Convergence is guaranteed if all (s,a) pairs visited infinitely + **SSAC** apply.

6.4 Off-Policy TD(0) Control: Q-Learning

Q-Learning uses the incremental update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$
Initialize $Q(s,a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S,A)\right]$
    $S \leftarrow S'$
  until S is terminal

Convergence is guaranteed as long as all pairs continue to be updated and a variant of the usual stochastic approximation conditions apply.

Notes:

1. The "$\max_a Q(s', a)$" is to eventually derive the optimal target policy
2. The behavioral policy for both TD algorithms uses ε-greedy to ensure that there is exploration
3. Notice how there is no **IS** in **Q-Learning**. This is because the a in "$\max_a Q(s', a)$" is not a random variable! The target policy is deterministic here.

6.5 Expected Sarsa (ESarsa)

This is a learning algorithm similar to **Q-Learning**, except it uses:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t)\right]$$
$$= Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

That is, it picks the maximum over next state-action pairs through expectation, considering the likelihood of each action under the current policy. Given the next state, S_{t+1}, this algorithm moves *deterministically* in the same direction as **Sarsa** moves *in expectation* (hence the name).
✓ reduces variance through random selection of A_{t+1} × more computationally complex than **Sarsa**

Sensitivity to α

The target in **ESarsa** is given as:

$$T_i = R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a)$$

The target is deterministic, i.e. $T_i \approx Q(S_t, A_t)$, no matter the step-size α. For **Sarsa**, as it samples A_{t+1}, it might have an action-value that is far from expectation and this is made worse with a larger step-size α.

On- or Off-Policy?

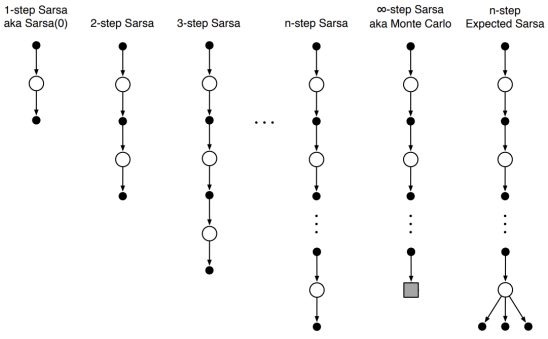
ESarsa could be either. If it is *off-policy* and the target policy is greedy + behavioral policy is ε-greedy then we exactly get **Q-Learning**.

6.6 N-Step TD Prediction

This is a combination of both **MC** and **TD**. The difference is below:

$$G_{t:t+1} \stackrel{\text{def}}{=} R_{t+1} + \gamma V_{t(S_{t+1})} \quad \text{TD(0) 1-Step Return}$$
$$G_{t:\infty} \stackrel{\text{def}}{=} \sum_{k+1}^{\infty} \gamma^{k-1} R_{t+k} \quad \text{MC Full Return}$$
$$G_{t:t+n} \stackrel{\text{def}}{=} \sum_{k+1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n V_{t+n-1}(S_t) \quad \text{N-Step TD}$$

That is, **N-Step TD** bootstraps over multiple steps.



The *n-step- update* is still TD as it changes an earlier estimate based on how it differs from a later estimate. The later estimate now is just n-steps away. The incremental value function update is:

$$V_{t+n}(S_t) \stackrel{\text{def}}{=} V_{t+n-1}(S_t) + \alpha\left[G_{t:t+n} - V_{t+n-1}(S_t)\right], \quad 0 \leq t < T$$

All updates begin from time $t+n$, when R_{t+n} is first known. Hence for the first $n-1$ steps, there are no changes made in each episode, but an equal number of additional updates are made at the end of the episode after termination but before starting on the next episode.

Note: If $t+n \geq T$, then $G_{t:t+n} = G_t$

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then $T \leftarrow t+1$
    $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha\left[G - V(S_\tau)\right]$
  Until $\tau = T - 1$

Notes:

1. All load and stores are done in a N-sized array, hence the mod
2. $G_{t:t+n}$ is calculated as long as $\tau + n < T$. As soon as $\tau + n = T$, exactly $n-1$ more steps are are taken (till $\tau = T-1$). Within those steps, the missing rewards R_{t+n} are just ignored ("corrected for").

Error-Reduction Property: The worst error of the expected *n-step* return is guaranteed to be $\leq \gamma^n$ times the worst error under V_{t+n-1}:

$$\max_s \left|\, \mathbb{E}_\pi[G_{t:t+n} \mid S_t = s] - v_\pi(s) \,\right| \leq \gamma^n \max_s \left|\, V_{t+n-1}(s) - v_\pi(s) \,\right| \quad \forall n \geq 1$$

Convergence of **n-step** is guaranteed given the *error reduction property*.

6.7 On-Policy N-Step for TD Control

For **on-policy n-step**, the action-value will have the form:

$$Q_{t+n} \stackrel{\text{def}}{=} Q_{t+n-1}(S_t, A_t) + \alpha\left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\right]$$

Only the future **return** term changes depending on the methods:

1. **N-Step On-Policy Sarsa**:

$$G_{t:t+n} \stackrel{\text{def}}{=} \sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n}), n \geq 1, 0 \leq t < T - n$$

where $G_{t:t+n} = G_t$ if $t + n \geq T$.

n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then:
        $T \leftarrow t+1$
      else:
        Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
    $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\left[G - Q(S_\tau, A_\tau)\right]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T - 1$

2. **N-Step On-Policy ESarsa**:

$$G_{t:t+n} \stackrel{\text{def}}{=} \sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n \overline{V}_{t+n-1}(S_{t+n}, A_{t+n}), t + n < T$$

where $\overline{V}$ is the **expected approximate value** of state s:

$$\overline{V}_t(s) \stackrel{\text{def}}{=} \sum_a \pi(a \mid s) Q_t(s,a) \text{ and } \overline{V}_t(\text{terminal}) = 0$$

6.8 Off-Policy N-Step for TD Control

For **off-policy n-step**, the state-value will have the form:

$$V_{t+n} \overset{\text{def}}{=} V_{t+n-1}(S_t) + \alpha\rho_{t:t+n-1}[G_{t:t+n} - V_{t+n-1}(S_t)], 0 \leq t < T$$

where the **IS ratio** is slightly modified to be:

$$\rho_{t:h} \overset{\text{def}}{=} \prod_{k=t}^{\min(h,T-1)} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

Between **Sarsa** and **ESarsa**, only the **IS ratio** changes as shown:

1. **N-Step Off-Policy Sarsa**

$$Q_{t+n} \overset{\text{def}}{=} Q_{t+n-1}(S_t, A_t) + \alpha\rho_{t+1:t+n}[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)]$$

2. **N-Step Off-Policy ESarsa**

$$Q_{t+n} \overset{\text{def}}{=} Q_{t+n-1}(S_t, A_t) + \alpha\rho_{t+1:t+n-1}[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)]$$

(as all last possible actions are taken in account under π)

The **IS ratio** helps to balance out the importance of certain actions:

1. If $\pi(a \mid s) = 0$, then no weight is to that action and it is ignored
2. If $\pi(a \mid s) > b(a \mid s)$, then a higher weight is given to the action as it could be an action common in π but rare in b. If the weighting was greater in b, the opposite would hold true.
3. If $\pi(a \mid s) = b(a \mid s)$, the ratio is 1 and there are no changes

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim b(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | | Take action A_t
  | | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | | If S_{t+1} is terminal, then:
  | | | $T \leftarrow t+1$
  | | else:
  | | | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:\tau+n}$)
  | | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
  | | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
  | | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho[G - Q(S_\tau, A_\tau)]$
  | | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
  Until $\tau = T - 1$

7 Planning and Learning

Planning: $\overset{\text{def}}{=}$ any process that uses a model of the environment to compute a plan of action (policy) to achieve a specified goal.

Types of Models:

1. **Distribution Model:** Produce a description of all possibilities and their probabilities. **DP** methods use these: $p(s', r \mid s, a)$
2. **Simulation (Sample) Model:** Produces sample outcomes $(s', r) \sim \hat{p}(s,a) s.t. \Pr\{\hat{p}(s,a) = (s', r)\} = p(s', r \mid s, a)$

Types of Planning:

1. **State-Space (SSP):** involves searching through the state space for an optimal policy or an optimal path to the goal. (we focus on thiws)
2. **Plan-Space:** involves searching through a space of plans instead.

SSP involves:

1. Computing value functions as a key step to improving the policy
2. Computing value function by updates or backup operations (operations around update targets) applies to simulated experience

Offline Planning:

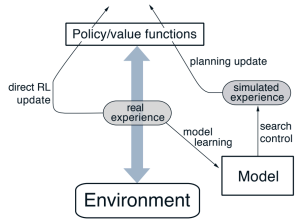
- Uses MDP (provided) to find optimal policy before applying to real experience + uses as much time as needed to find optimal policy
- Policy is **complete** - it can give an optimal action for all possible states.
- Examples: **DP** and **Dyna-Q**

Online Planning:

- Uses an MDP to find the best policy as the MDP is experienced
- Limited compute budget at each state
- Policy usually incomplete: only possible to compute optimal action for current state
- Examples: **Rollout Planning** including **MCTS**.

7.1 Dyna-Q

Dyna: RL architecture that includes planning, learning and acting



Direct RL: Involves methods like **MC** or **TD** that learn from experience
Search Control: Refers to the process that selects starting states and actions for the simulated experiences.
Dyna-Q: This is where **Q-Learning** is used for both learning and planning. It is possible for the planning, acting and model-learning steps of **Dyna** to be carried out in parallel but the provided algorithm is a sequential one:

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
  (a) $S \leftarrow$ current (nonterminal) state
  (b) $A \leftarrow \varepsilon$-greedy(S,Q)
  (c) Take action A; observe resultant reward, R, and state, S'
  (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
  (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
  (f) Loop repeat n times:
    $S \leftarrow$ random previously observed state
    $A \leftarrow$ random action previously taken in S
    $R, S' \leftarrow Model(S,A)$
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

Note:

1. Direct RL, model-learning and planning are implemented by steps (d), (e), and (f) respectively.
2. The planning step involves *random-sample one-step Q-Learning*, where the estimate Q is improved **only** through prior experienced (s,a) pairs!
3. The environment is assumed too be deterministic.
4. Updating the value function through planning is called **indirect RL**.

What happens when model is wrong?

The learned model could be wrong when:

1. Stochastic environment but only limited number of samples observed
2. Model learned using approximation that has generalized poorly
3. Environment has changed and new behavior has not been observed

In some cases, suboptimal policy computed by planning quickly leads to the discovery and correction of the modeling error. This tends to happen when the model is optimistic in the sense of predicting greater reward or better state transitions than are actually possible. The planned policy attempts to exploit these opportunities and in doing so discovers that they do not exist.

Dyna-Q+

In order to improve exploration in Dyna-Q, a heuristic is introduced to force the agent to try (s,a) pairs that is has not for a long time by forcing it to assume that the model for them is incorrect. This is achieved simply by adding a *bonus reward* to (s,a), scaling with how long ago they were tried:

$$R'_t = R_t + k\sqrt{\tau}$$

where k is small and τ refers the time since last visiting (s,a).

7.2 Rollout Algorithms

1. *Dyna-Q* uses a model to reuse past experiences. *Rollout* algorithms instead use a model to simulate ("rollout") future trajectories.
2. They apply **MC** control to simulated trajectories that all being at the current environment state.
3. They estimate action values for a given policy by averaging the returns of many simulated trajectories that start with each possible action and then follow the given policy. When the action-value estimates are considered to be accurate enough, the action (or one of the actions) having the highest estimated value is executed, after which the process is carried out anew from the resulting next state
4. Goal is not to estimate a *complete optimal policy* but instead to produce **MC** estimates of action-values **only** for each current state using a *rollout policy*, making immediate use of the estimates and then discarding them.
5. The *policy improvement theorem* holds with this approach through results similar to one step of policy iteration (closer to *asynchronous DP*).

Rollout Q-planning with forward updating:

Given: simulation model *Model*
Initialise: $Q(s,a)$ for all s, a
for $t = 0, 1, 2, 3, \ldots$ do
  $S_t \leftarrow$ current state
  for n rollouts do
    $S \leftarrow S_t$
    while S is non-terminal (or fixed-length rollouts) do
      select action A based on $Q(S,\cdot)$ with some exploration // *e.g. ϵ-greedy*
      $(R, S') \sim Model(S,A)$
      Q-update: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
      $S \leftarrow S'$
  select action A_t greedily from $Q(S_t,\cdot)$

Rollout Q-planning with backward updating:

Given: simulation model *Model*
Initialise: $Q(s,a)$ for all s, a; LIFO stack *Trace* $= \{\}$
for $t = 0, 1, 2, 3, \ldots$ do
  $S_t \leftarrow$ current state
  for n rollouts do
    $S \leftarrow S_t$
    while S is non-terminal (or fixed-length rollouts) do // *Rollout*
      select action A based on $Q(S,\cdot)$ with some exploration
      $(R, S') \sim Model(S,A)$
      push (S, A, R, S') to *Trace*
      $S \leftarrow S'$
    while *Trace* not empty do // *Backprop*
      pop (S, A, R, S') from *Trace*
      $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
  select action A_t greedily from $Q(S_t,\cdot)$

× Several timing challenges with Rollout: number of actions that have to be evaluated for each decision, the number of time steps in the simulated trajectories needed to obtain useful sample returns, the time it takes the rollout policy to make decisions, and the number of simulated trajectories needed to obtain good Monte Carlo action-value estimates. ✓ **MC** trials are independent can can be ran in parallel ✓ simulated trajectories can be truncated ✓ actions unlikely to be the best can be pruned away

7.3 MC Tree Search (MCTS)

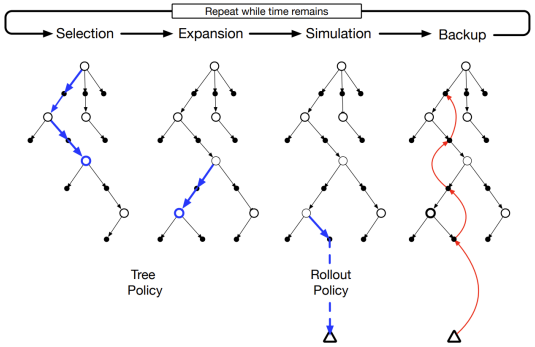
1. General, efficient rollout planning with *backward updating*
2. Stores partial Q as tree and asymmetrically expand tree based on most promising actions. That is,

$$Q(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') \mid S_t = a, A_t = a\Big]$$

3. Like any **MC** method, value of (s,a) pairs are estimated as average simulated returns. However, estimates for only a subset of promising (s,a) pairs are kept in a tree, rooted from the current state.
4. **MCTS** incrementally extends the tree. Outside the tree and at leaf nodes, a *rollout policy* is used. At states inside the tree, action are picked from the stored estimates, giving a *tree policy*.

Phases of MCTS:

1. **Selection:** Starting at the root, the *tree policy* is used to traverse the tree and select an action at one the leaf nodes.
2. **Expansion:** *On some iterations*, the tree is expanded from the selected leaf node by adding ≥ 1 child nodes, reached via unexplored actions.
3. **Simulation:** From the selected leaf node, or one of its newly-added children, episode simulate is ran using the *rollout policy*. The result is a **MC** trial with actions selected first by the *tree policy* and beyond the tree by the *rollout policy*.
4. **Backup:** The return generated is backed up to update, or to initialize, the action values attached to the edges of the tree traversed by the *tree policy* in this iteration of **MCTS**. No values are saved for the states and actions visited by the *rollout policy* beyond the tree.



MCTS continues executing these four steps, starting each time at the tree's root node, until no more time is left, or some other computational resource is exhausted. An action is selected from the actions right after the root and **MCTS** is run again using the subtree rooted under the selected action or completely from scratch (not done often).

MCTS-Search(S_t):

Find node v_0 with $state(v_0) = S_t$ (or create new node)
while within computational budget do
  $v_l \leftarrow TreePolicy(v_0)$ // *Select node in tree and expand*
  $\Delta \leftarrow DefaultPolicy(state(v_l))$ // *Simulation steps*
  $Backprop(v_l, \Delta)$
return $action(BestChild(v_0))$ // *e.g. highest expected return; most visited child*

UCB for Trees (UCT)

The *tree policy* is meant to be exploratory, so it could be ε-greedy but it could also use **UCB** as follows:

$$A := \begin{cases} a \text{ if } a \text{ never tried in } s \\ \text{argmax}_a Q(s,a) + c\sqrt{\log\left(\frac{N(s)}{N(s,a)}\right)} \text{ if } a \neq A^* \end{cases}$$

where $N(s)$ is number of times s has been visited and $N(s,a)$ is the number of times a was selected in s.

Simulation step gives estimate of return at state, e.g.:

Random-DefaultPolicy(S):

$G \leftarrow 0$
while S is non-terminal do
  $A \leftarrow$ random action (uniformly)
  $(R, S') \sim Model(S,A)$
  $G \leftarrow R + \gamma G$
  $S \leftarrow S'$
return G

Possible improvements:

- Average over multiple simulations
- Use domain-specific heuristic to
  - select better actions than random
  - evaluate state directly (e.g. in Chess we know that some states are better than others)

8 Value Function Approximation

Curse of Dimensionality: The number of states grow *exponentially* with the number of state variables k, i.e. $O(n^k)$ states.
Approximation allows us to avoid the above and does not assume 1. we have *unlimited space* and 2. *unlimited data*. **We first look at on-policy.**
Idea: Use function approximation of the v_π by representing it in a parametrized functional form with a weight vector $w \in \mathbb{R}^d$:

$$\hat{v}(s, \boldsymbol{w}) \approx v_\pi(s)$$

The weight vector $\boldsymbol{w}$ could represent feature weights, connection weights of a neural net, or it could be numbers defining the split points and leaf values of a decision tree. Typically $d << | \mathcal{S} |$ and changing one weight changes estimated values of many states (thus generalizing).

Advantages of Approximation

1. If $\hat{v}$ does not capture certain aspects of a state, it is almost like those aspects are not observed. Hence, functional approximation allows RL to apply to partially observable problems too!
2. Tabular methods require storage proportional to $| \mathcal{S} |$ or $| \mathcal{S} | * | \mathcal{A} |$ - $\hat{v}$ does not and is **compact**.
3. Other methods suffer from not having any or having insufficient data for estimation. $\hat{v}$ **generalizes** to unknown (s, a) as a fix.

Supervised Learning

Learning a value function from (input, output) pairs is a form of supervised learning which RL also utilizes. Consider pairs of states and their update targets (S_t, U_t) we have looked at so far:

1. **MC**: $U_t = G_t$
2. **TD(0)**: $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \boldsymbol{w})$
3. **n-step TD**: $U_t = \sum_{k=1}^{n-1} \gamma^{k-1} R_{t+1} + \gamma^n \hat{v}(S_{t+n}, \boldsymbol{w}_{t+n-1})$

The desired properties of supervised learning are:

1. **Incremental Updates** (update $\boldsymbol{w}$ using only partial data)
2. **Ability to handle noisy targets**
3. **Ability to handle non-stationary targets**

If $\hat{v}$ or $\hat{q}$ were differentiable, **stochastic gradient descent (SGD)** suits.

Prediction Objective

Important to clarify as our assumption is that there are far more states than weights, so making one state's estimate more accurate invariably means making other's less accurate
Let $\mu(s) \geq 0, \forall s \in S$ represent how much we "care" about a particular state. Now, we consider the error between $\hat{v}(s, \boldsymbol{w})$ and $v_\pi(s)$ as:
Mean Squared Error : A continuous measure of prediction quality.

$$\overline{\text{VE}}(\boldsymbol{w}) \stackrel{\text{def}}{=} \sum_s \mu(s)[v_\pi(s) - \hat{v}(s, \boldsymbol{w})]^2$$

Often $\mu(s)$ is chosen as the fraction of time spent in s (i.e. *on-policy distribution*). Goal is to find $\boldsymbol{w}^*$ that minimizes $\overline{\text{VE}}$. Finding a *global optimum* that does so is difficult, so often it is sufficient to seek to converge to just a *local optimum*.

8.1 Stochastic Gradient and Semi-Gradient Methods

Let $\boldsymbol{w}$ be a column vector of weights and let $\hat{v}(s, \boldsymbol{w})$ be a differentiable function of $\boldsymbol{w}, \forall s \in S$. Then, *gradient descent* can be used to find the *local minimum* iteratively as follows:

$$\begin{aligned}\boldsymbol{w}_{t+1} &\stackrel{\text{def}}{=} \boldsymbol{w}_t - \frac{1}{2}\alpha \nabla [v_\pi(S_t) - \hat{v}(S_t, \boldsymbol{w}_t)]^2 \\ &= w_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \boldsymbol{w}_t)] \nabla \hat{v}(S_t, \boldsymbol{w}_t)\end{aligned}$$

where $\alpha > 0$ and $\nabla f(\boldsymbol{w})$ denotes a column vector of partial derivatives:

$$\nabla f(\boldsymbol{w}) \stackrel{\text{def}}{=} \left(\frac{\partial f(\boldsymbol{w})}{\partial \boldsymbol{w}_1}, \frac{\partial f(\boldsymbol{w})}{\partial \boldsymbol{w}_2}, ..., \frac{\partial f(\boldsymbol{w})}{\partial \boldsymbol{w}_d} \right)$$

Gradient descent methods are stochastic and at each step, minimize $\overline{\text{VE}}$.
Convergence of **SGD** assumes that α decreases over time + **SSAC** apply.
However, $v_\pi(S_t)$ is unknown, so use expected *update target* U_t:

$$\boldsymbol{w}_{t+1} = w_t + \alpha [U_t - \hat{v}(S_t, \boldsymbol{w}_t)] \nabla \hat{v}(S_t, \boldsymbol{w}_t)$$

If U_t is unbiased, i.e. $\mathbb{E}[U_t \mid S_t = s] = v_\pi(s)$, then $\boldsymbol{w}_t$ is guaranteed to converge to a local optimum given stochastic approximation conditions on a decreasing α. **SGD** is optimal for **MC** as they produce an unbiased estimate of v_π. For **bootstrapping** methods it is biased:

1. **TD** target $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \boldsymbol{w})$
2. **DP** target: $U_t = \sum_{a,s',r} \pi(a \mid S_t) p(s', r \mid s, a)[r + \gamma \hat{v}(s', \boldsymbol{w})]$
3. **N-Step target**: $U_t = \sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n \hat{v}(S_{t+n}, \boldsymbol{w}_{t+n})$

All the above use $\boldsymbol{w}$ and ignore its effect on the target. Hence, they are called **semi-gradient methods** as it is not true **gradient descent**.

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[G_t - \hat{v}(S_t, \mathbf{w})\big] \nabla \hat{v}(S_t, \mathbf{w})$

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\big] \nabla \hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

State Aggregation

It is a simple form of generalizing function approximation in which states are grouped together, with one estimated value for all states in a group. The value of a state is estimated as its group's component, and updated when the state is. *State Aggregation* is a special case of **SGD** in which the gradient $\nabla \hat{v}(S_t, \boldsymbol{w}_t) = 1$ for S_t's group's component and 0 for the rest.

Linear Methods

Where $\hat{v}$ is a linear function:

$$\hat{v}(s, \boldsymbol{w}) \stackrel{\text{def}}{=} \boldsymbol{w}^T \boldsymbol{x}(s) \stackrel{\text{def}}{=} \sum_{i=1}^{d} \boldsymbol{w}_i \boldsymbol{x}_i(s)$$

Where $\boldsymbol{x}(s)$ is the *feature vector* representing s. In the linear case, the features form a *linear basis* for the set of approximate functions. The linear case has the following important result:

$$\nabla \hat{v}(s, \boldsymbol{w}) = \boldsymbol{x}(s)$$

Hence the **SGD** update for the linear case is simply:

$$\boldsymbol{w}_{t+1} = w_t + \alpha [U_t - \hat{v}(S_t, \boldsymbol{w}_t)] \boldsymbol{x}(S_t)$$

Moreover, there is only a single optimum in the linear case, so linear methods, when they converge, converge to or near the *global optimum*. The **MC** gradient updates converge to the *global* optimum, whereas the **TD** gradient update converge *near global optimum*.

8.2 Coarse/Tile Coding

Both come under *feature construction* for linear methods and both allow for generalization across disjoint sets of states unlike *state aggregation* with generalizes within a particular set. *Note* that linear methods cannot take into account the interaction between features!

Coarse Coding

Assuming $S \in \mathbb{R}^2$ + binary features, the state space is represented by **overlapping circles** corresponding to features. Each state falls within some number of circles. Any update to the state, updates the features corresponding to those circles and thereby all other states in those circles.

Tile Coding

These allow *coarse coding* for multi-dimensional spaces. The state space is split into partitions, where each partition is called a *tiling* and each element of the partition is a *tile*. Each state has exactly the same number of features active as the number of tilings (each *tile* can be thought of as a grid of features). $\boldsymbol{x}(s)$ here has one component for each tile.
Generalization occurs to states other than the one trained if those states fall within any of the same tiles, proportional to the number of states in common.

8.3 On-Policy Control in Episodic Tasks w/ Approximation

State-action values estimated by: $\hat{q}(s, a, \boldsymbol{w}) \approx q_\pi(s, a)$.
Linear approximation: $\hat{q}(s, a, \boldsymbol{w}) \stackrel{\text{def}}{=} \sum_{i=1}^{d} \boldsymbol{w}_i \boldsymbol{x}_i(s, a)$
SGD with action-value:

$$\boldsymbol{w}_{t+1} = w_t + \alpha [U_t - \hat{q}(S_t, A_t, \boldsymbol{w}_t)] \nabla \hat{q}(S_t, A_t, \boldsymbol{w}_t)$$

Update targets for **TD** control methods as below:

1. **Sarsa:** $U_t = R_{t+1} \gamma \hat{q}(S_{t+1}, A_{t+1}, \boldsymbol{w}_t)$
2. **Q-Learning:** $U_t = R_{t+1} \gamma \max_a \hat{q}(S_{t+1}, a, \boldsymbol{w}_t)$
3. **ESarsa:** $U_t = R_{t+1} \gamma \sum_a \pi(a \mid S_{t+1}) \hat{q}(S_{t+1}, a, \boldsymbol{w}_t)$

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[R - \hat{q}(S, A, \mathbf{w})\big] \nabla \hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big] \nabla \hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

Convergence to Global Optimum in Episodic Control

Algorithm	Tabular	Linear	Non-Linear
MC Control	Yes	Chatter	No
Semi-Gradient n-step Sarsa	Yes	Chatter	No
Semi-Gradient n-step Q-Learning	Yes	No	No

where *chatter* refers to being near optimal because optimal policy may not be representable under value function approximation

Deadly Triad

Refers to **risk of divergence** rising when the following three are combined:
Function Approximation + **Bootstrapping** + **Off-Policy Learning**
Possible fixes:

1. Use **IS** to warp off-policy distribution into on-policy distribution
2. Use gradient TD methods which follow true gradient of projected Bellman error

9 Policy Gradient Methods

Three kinds of RL Algorithms:

1. **Value-Based**: use estimates Q, V to find optimal policy.
2. **Model-Based:** learn model through approximations $\hat{q}, \hat{r}$ or have a perfect model like **DP** methods, and **plan** to get optimal policy
3. **Policy-Based:** Learn policy directly!

Policy-Based RL

Let $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ be some policy parameter vector such that

$$\pi(a \mid s, \boldsymbol{\theta}) = \Pr\{A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}\}$$

If a method uses a learned value function, then its weight vector is denoted $\boldsymbol{w} \in \mathbb{R}^d$, as usual, in $\hat{v}(s, \boldsymbol{w})$. Let $J(\boldsymbol{\theta})$ denote the scalar performance measure w.r.t. the policy parameter. These methods seek to maximize performance using *gradient ascent* as follows:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}$$

where $\widehat{\nabla J(\boldsymbol{\theta}_t)} \in \mathbb{R}^{d'}$ is a stochastic estimate whose expectation approximates the gradient of the performance measure w.r.t. $\boldsymbol{\theta}_t$.
Note: Methods that learn approximations to both policy and value functions are often called *actor–critic* methods. *Actor* refers to the learned policy, and *Critic* refers to the learned value function, usually a state-value function.

9.1 Representing the Policy

The policy can be parametrized in any way, as long as $\pi(a \mid s, \boldsymbol{\theta})$ is differentiable w.r.t. its parameters. To ensure exploration, we ensure that the policy is never becomes deterministic.

Soft-Max Representation:

$$\pi(a \mid s, \boldsymbol{\theta}) = \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_{b \in A} e^{h(s,b,\boldsymbol{\theta})}}$$

h is a preference function (recall **Gradient Bandits** in 2.2) and can be given linearly as $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{x}(s, a)$ where $\boldsymbol{x}(s, a)$ is a feature vector $\in \mathbb{R}^{d'}$.
✓ The approximate policy approaches a deterministic policy unlike a ε-greedy policy ✓ Enables the selection of actions with arbitrary probabilities

Gaussian Representation:

$$\pi(a \mid s, \boldsymbol{\theta}) = N(\mu(s, \boldsymbol{\theta}), \sigma^2) \text{ and } \mu(s, \boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{x}(s)$$

9.2 Policy-Optimisation Problem

Problem: Given $\pi(a \mid s)$, interact with **MDP** M and find optimal $\boldsymbol{\theta}$
We limit our problems to *episodic tasks* and solve them using a performance measure $J(\boldsymbol{\theta})$, defined as the value of the **start state** of the episode:

$$J(\boldsymbol{\theta}) \stackrel{\text{def}}{=} v_{\pi_{\boldsymbol{\theta}}}(s_0)$$

Issue with $J(\boldsymbol{\theta})$ is that performance depends on both action selection and the distribution of states in which those selections are made (both affected by $\boldsymbol{\theta}$). Computing the effect of $\boldsymbol{\theta}$ on actions, and thus rewards, is simple. However, the effect of $\boldsymbol{\theta}$ on state distribution requires knowledge of the environment, which is assumed to be unknown. The following helps:
Policy Gradient Theorem:
For any differentiable policy π, the policy gradient is:

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s d_\pi(s) \sum_a q_\pi(s, a) \nabla \pi(a \mid s, \boldsymbol{\theta})$$

where $d_\pi(s)$ is the *on-policy distribution* under π. In *episodic tasks*, the above equation is actually an equality, but the proportional case includes continuing tasks. The *on-policy distribution* can be:

1. Using *start-state value*: $d_\pi(s) = \sum_{t=0}^{\infty} \gamma^t \Pr\{S_t = s \mid s_0, \pi\}$
2. Using *average reward*: $d_\pi(s) = \lim_{t \to \infty} \Pr\{S_t = s \mid \pi\}$

Note: No environment dynamics involved at all!

9.3 Policy-Gradient Theorem Breakdown

The theorem is useful as it transforms the earlier *gradient ascent* rule using $\widehat{\nabla J(\boldsymbol{\theta}_t)}$ into a simpler update rule. The derivation is provided below:

$$\begin{aligned}\nabla J(\boldsymbol{\theta}) &= \sum_s d_\pi(s) \sum_a q_\pi(s, a) \nabla \pi(a \mid s, \boldsymbol{\theta}) \\ &\text{replace s by sample } S_t \sim \pi \\ &= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a \mid S_t, \boldsymbol{\theta}) \right] \\ &\text{introduce the policy} \\ &= \mathbb{E}_\pi \left[\sum_a \pi(a \mid S_t, \boldsymbol{\theta}) q_\pi(S_t, a) \frac{\nabla \pi(a \mid S_t, \boldsymbol{\theta})}{\pi(a \mid S_t, \boldsymbol{\theta})} \right] \\ &\text{replace a by sample } A_t \sim \pi \\ &= \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta})}{\pi(A_t \mid S_t, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta})}{\pi(A_t \mid S_t, \boldsymbol{\theta})} \right]\end{aligned}$$

$$= \mathbb{E}_\pi[G_t \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta})] \text{ (using } \nabla \ln x = \nabla \frac{x}{x})$$

Finally, we get the update rule (called **REINFORCE**):

$$\theta_{t+1} \stackrel{\text{def}}{=} \theta_t + \alpha \left[G_t \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta})}{\pi(A_t \mid S_t, \boldsymbol{\theta})} \right] \text{ or } \theta_{t+1} \stackrel{\text{def}}{=} \theta_t + \alpha G_t \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta})$$

The vector is the direction in parameter space that most increases the probability of repeating the action A_t on future visits to state S_t. The update increases the parameter vector in this direction proportional to the return, and inversely proportional to the action probability. The former makes sense because it causes the parameter to move most in the directions that favor actions that yield the highest return. The latter makes sense because otherwise actions that are selected frequently are at an advantage (the updates will be more often in their direction) and might win out even if they do not yield the highest return.
Algorithms that use **REINFORCE** typically approximate the terms q_π and $\nabla \ln \pi$. **MC** estimates can be used for q_π (as shown below) whereas $\nabla \ln \pi$ can be estimated as:

$$\nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t) := \begin{cases} \boldsymbol{x}(s,a) - \sum_{a'} \pi(a' \mid s, \boldsymbol{\theta}) \boldsymbol{x}(s,a') \text{ if using softmax} \\ (a - \mu(s,\boldsymbol{\theta})) \frac{\boldsymbol{x}(s)}{\sigma^2} \text{ if using Gaussian} \end{cases}$$

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\quad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

✓ The expected update over an episode is in the same direction as the performance gradient. This assures an improvement in expected performance for suciently small α, and convergence to a local optimum under **SSAC** for decreasing α **×** As a **MC** method, REINFORCE may be of high variance and thus produce slow learning.

REINFORCE w/ Baseline

An approach to mitigate variance is to add a baseline $b(s)$ that is deducted from returns and does not change expectation.

$$\theta_{t+1} = \theta_t + \alpha[q_\pi(S_t, A_t) - b(S_t)] \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t)$$
$$\text{typically: } b_{S_t} = \hat{v}(S_t)$$

The baseline $b(S_t)$ can be any function as long as t does not vary with a as:

$$\theta_{t+1} = \theta_t + \alpha[q_\pi(S_t, A_t) - b(S_t)] \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t)$$
$$\text{adding back implicit expectation}$$
$$\theta_{t+1} = \theta_t + \alpha[\mathbb{E}(q_\pi(S_t, A_t) \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t)) -$$
$$\mathbb{E}(b(S_t) \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t))]$$
$$\text{we show that the second term above is 0 in expectation}$$
$$\mathbb{E}(b(S_t) \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t))$$
$$= \sum_a \pi(a \mid S_t, \boldsymbol{\theta}) b(S_t) \frac{\nabla \pi(a \mid S_t, \boldsymbol{\theta})}{\pi(a \mid S_t, \boldsymbol{\theta})}$$
$$= \sum_a b(S_t) \nabla \pi(a \mid S_t, \boldsymbol{\theta}) = b(S_t) \nabla \sum_a \pi(a \mid S_t, \boldsymbol{\theta}) = b(S_t) \nabla 1 = 0$$

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\quad (G_t)$
    $\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

9.4 Actor-Critic Methods

In **REINFORCE** w/ baseline, the learned state-value function estimates the value of the *first* state of each state transition. In actor–critic methods, on the other hand, the state-value function is applied also to the second state of the transition. That is, target becomes $G_{t:t+1}$ (like **TD(0)**).
Actor-Critic Update:

$$\theta_{t+1} = \theta_t + \alpha[R_{t+1} + \gamma \hat{v}(S_{t+1}, \boldsymbol{w}) - \hat{v}(S_t, \boldsymbol{w})] \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t)$$

Critic: Gives the state-value function estimate $\hat{v}(S_t, \boldsymbol{w})$
Actor: Gives the policy estimate $\pi(A_t \mid S_t, \boldsymbol{\theta}_t)$
The main appeal of one-step methods is that they are fully online and incremental. Note also that these methods lie between *policy-methods* and *value-methods*.

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S, \boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ $\quad$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S, \boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

10 Exam Tips/FAQ

Add your FAQ here!